Error Handling

Chapter 12

Randy Connolly and Ricardo Hoar

© 2015 Pearson

Section 1 of 6

WHAT ARE ERRORS AND EXCEPTIONS?

Types of Errors

- **Expected errors**

  Things that you expect to go wrong. Bad user input, database connection, etc…

- **Programming errors**

  Using undeclared variables, wrong names of functions during calls, etc...

- **Warnings and Notices**

  Problems that generate a PHP warning message but will not halt the execution of the page

- **Fatal errors**

  are serious in that the execution of the page will terminate unless handled in some way

Exceptions vs Errors

Not the same thing

- A **warning**, **notice**, or **error** is some type of problem that generates a nonfatal warning message or that generates an error message that terminates the program's execution.
- An **exception** refers to objects that are of type Exception and which are used in conjunction with the object-oriented **try** . . . **catch** language construct for dealing with runtime errors.

Section 2 of 6

PHP ERROR REPORTING

PHP error reporting

Lots of control

PHP has a flexible and customizable system for reporting warnings and errors that can be set programmatically at runtime or declaratively at design-time within the **php.ini** file.

There are three main error reporting flags:

- **error_reporting**
- **display_errors**
- **log_errors**

The error_reporting setting

What is an error?

The **error_reporting** setting specifies which type of errors are to be reported.

It can be set programmatically inside **any** PHP file:

```
error_reporting(E_ALL);
```

It can also be set within the **php.ini** file:

```
error_reporting = E_ALL
```

The error_reporting setting

Some error reporting constants

Constant Name	Value	Description
E_ALL	8191	Report all errors and warnings
E_ERROR	1	Report all fatal runtime errors
E_WARNING	2	Report all nonfatal runtime errors (that is, warnings)
	0	No reporting

The display_errors setting

To show or not to show

The **display_error** setting specifies whether error messages should or should not be displayed in the browser.

It can be set programmatically via the **ini_set()** function:

```
ini_set('display_errors','0');
```

It can also be set within the **php.ini** file:

```
display_errors = Off
```

The log_error setting

To record or not to record

The **log_error** setting specifies whether error messages should or should not be sent to the server error log.

It can be set programmatically via the **ini_set()** function:

```
ini_set('log_errors','1');
```

It can also be set within the **php.ini** file:

```
log_errors = On
```

The log_error setting

Where to store.

The location to store logs in can be set programatically:

```
ini_set('error_log', '/restricted/my-errors.log');
```

It can also be set within the **php.ini** file:

```
error_log = /restricted/my-errors.log
```

The log_error setting

error_log()

You can also programmatically send messages to the error log at any time via the **error_log()** function

```
$msg = 'Some horrible error has occurred!';

// send message to system error log (default)
error_log($msg,0);

// email message
error_log($msg,1,'support@abc.com','From: somepage.php@abc.com');

// send message to file
error_log($msg,3, '/folder/somefile.log');
```

Section 3 of 6

PHP ERROR AND EXCEPTION HANDLING

Procedural Error Handling

Recall connecting to a database, that there may be an error…

```
$connection = mysqli_connect(DBHOST, DBUSER, DBPASS, DBNAME);

$error = mysqli_connect_error();
if ($error != null) {
   // handle the error
   ...
}
```

LISTING 12.2 Procedural approach to error handling

OO Exception Handling

Try, catch, finally

When a runtime error occurs, PHP *throws* an *exception*.

This exception can be *caught* and handled either by the function, class, or page that generated the exception or by the code that called the function or class.

If an exception is not caught, then eventually the PHP environment will handle it by terminating execution with an "Uncaught Exception" message.

OO Exception Handling

Try, catch, finally

```php
// Exception throwing function
function throwException($message = null,$code = null) {
  throw new Exception($message,$code);
}

try {
  // PHP code here
  $connection = mysqli_connect(DBHOST, DBUSER, DBPASS, DBNAME)
    or throwException("error");
 //...
}
catch (Exception $e) {
  echo ' Caught exception: ' .  $e->getMessage();
  echo ' On Line : ' .  $e->getLine();
  echo ' Stack Trace: '; print_r($e->getTrace());
} finally {
  // PHP code here that will be executed after try or after catch
}
```

LISTING 12.3 Example of try . . . catch block

OO Exception Handling

Finally

The **finally** block is optional. Any code within it will always be executed *after* the code in the try or in the catch blocks, even if that code contains a return statement.

The finally block is only available in PHP 5.5 and later

OO Exception Handling

```
// foo() will never get called as those operators
// are short-circuit
$a = (false && foo());
$b = (true || foo());
$c = (false and foo());
$d = (true or foo());

// "||" has a greater precedence than "or"
// The result of the expression (false || true) is
// assigned to $e
// Acts like: ($e = (false || true))
$e = false || true;
// The constant false is assigned to $f before
// the "or" operation occurs
// Acts like: (($f = false) or true)
$f = false or true;
// same for &&,and
```

***

print_r() displays information about a variable in a way that's readable by humans.

Throw your own exception

Object oriented way of dealing with the unexpected

```
try {
  // PHP code here
}
catch (Exception $e) {
    // do some application-specific exception handling here
    ...
    // now rethrow exception
    throw $e;
}
```

LISTING 12.5 Rethrowing an exception

Custom Handlers

Error and Exception Handlers

What should a custom error or exception handler do?

It should provide the *developer* with detailed information about the state of the application when the exception occurred, information about the exception, and when it happened.

It should hide any of those details from the regular end user, and instead provide the user with a generic message such as “Sorry but there was a problem”

Once a handler function is defined, it must be registered, using the following code:

```
set_exception_handler('my_exception_handler');
```

the same can be done for errors using a similar function.

Custom Handlers

Error and Exception Handlers

```
function my_exception_handler($exception) {

  // put together a detailed exception message
  $msg = "<p>Exception Number " . $exception->getCode();
  $msg .= $exception->getMessage() . " occurred on line ";
  $msg .= "<strong>" . $exception->getLine() . "</strong>";
  $msg .= "and in the file: ";
  $msg .= "<strong>" . $exception->getFile() . "</strong> </p>";

  // email error message to someone who cares about such things
  error_log($msg, 1, 'support@domain.com',
            'From: reporting@domain.com');

  // if exception serious then stop execution and tell maintenance fib
  if ($exception->getCode() !== E_NOTICE) {
     die("Sorry the system is down for maintenance. Please try
          again soon");
  }
}
```

LISTING 12.6 Custom exception handler

Example 1

```
<?php


// Using a non existing variable
echo $myvar;
echo "Hello!\n";

?>
```

$ php esempio1.php
Warning: Undefined variable $myvar in /Users/vecchio/Documents/didattica/PWEB/php/errori/esempio1.php on line 5
Hello!

Example 2

```php
<?php

// Calling a method on a non-existing variable
echo $myvar->a_method();
echo "Hello\n";

?>
```

Warning: Undefined variable $myvar in ...errori/esempio2.php on line 4

Fatal error: Uncaught Error: Call to a member function a_method() on null in ...errori/esempio2.php:4
Stack trace:
#0 {main}
  thrown in ..errori/esempio2.php on line 4

Example 3

```php
<?php

// Turn off error reporting
ini_set('error_reporting', 0);

// Warning message is suppressed
echo $myvar;
// Error message is suppressed
echo $myvar->a_method();
// The following line is not executed because of fatal error
// produced by the previous line
echo "Hello\n";

?>
```

No error messages are shown

Example 4

```php
<?php
function my_error_handler($errno, $errstr) {
  // My custom actions, here just printing
  echo "There was a problem. Error number: "
      . $errno . ", message: " . $errstr . "\n";
}
// Set a customized error handler
set_error_handler("my_error_handler", E_ALL);
// Turn off error reporting
ini_set('error_reporting', 0);
// Warning message is suppressed
echo $myvar;
// Error message is suppressed
echo $myvar->a_method();
// The following line is not executed because of fatal error
// produced by the previous line
echo "Hello\n";
?>
```

```
There was a problem. Error number: 2, message: Undefined variable $myvar
There was a problem. Error number: 2, message: Undefined variable $myvar
```

Example 5

```php
<?php
try {
  echo $myvar;
  echo $myvar->a_method();
  echo "Hello\n";
} catch (Exception $e) {
  echo "In the first catch block\n";
} catch (Error $e) {
  echo "In the second catch block\n";
  echo "Code: " . $e->getCode()
      . ", message: " . $e->getMessage()
      . ", line: " . $e->getLine()
      . ", file: " . $e->getFile() . "\n";
}
echo "Another hello!\n";
?>
```

```
Warning: Undefined variable $myvar in esempio5.php on line 3
Warning: Undefined variable $myvar in esempio5.php on line 4
In the second catch block
Code: 0, message: Call to a member function a_method() on null, line: 4, file: esempio5.php
Another hello!
```

Example 6

```php
<?php
try {
  echo $myvar;
  ( // <--
  echo "Hello\n";
} catch (Exception $e) {
  echo "In the first catch block\n";
} catch (Error $e) {
  echo "In the second catch block\n";
  echo "Code: " . $e->getCode()
      . ", message: " . $e->getMessage()
      . ", line: " . $e->getLine()
      . ", file: " . $e->getFile() . "\n";
}
echo "Another hello!\n";
?>
```

Parse error: syntax error, unexpected token "echo" in esempio6.php on line 6